SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I KESAVAN 1924260259 *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1: AI-POWERED EMERGENCY MEDICINE DELIVERY DRONES

i. Behaviour of Greedy Best-First Search, strengths and weaknesses

Greedy Best-First Search (GBFS) selects the node that appears closest to the destination according to a heuristic function. In this scenario, the heuristic is the straight-line distance between the drone's current position and the medicine delivery point.

Evaluation function: f(n) = h(n)

Here, h(n) estimates the remaining distance or cost to the destination. The drone prioritizes routes that appear direct. If it discovers a blocked path, it must update the available routes and search again.

Strengths

- Fast decision-making because it uses only the heuristic.
- Useful when emergency medicines must be delivered quickly.
- Can reduce the number of nodes explored when the heuristic is informative.

Weaknesses

- Does not consider the actual cost already incurred to reach a node.
- May choose a short but dangerous or expensive route.
- Does not guarantee an optimal route.
- Can perform poorly when information about blocked paths is incomplete.

ii. How A* handles the same situation

A* combines the actual cost of reaching a node with the estimated cost remaining to the destination.

Evaluation function: f(n) = g(n) + h(n)

g(n) is the actual cost from the starting point to node n; h(n) is the estimated remaining cost; and f(n) is the estimated total route cost. For the drone, g(n) may represent travel time, fuel consumption and weather-related penalties. A* considers both accumulated cost and estimated remaining cost. If a route becomes blocked, the drone can update its map and replan.

Advantages: It often finds better routes than GBFS, can find an optimal path when the required cost and heuristic conditions hold, and can incorporate terrain and weather costs. Limitation: It may require considerable memory and computation and may need to replan when the environment changes.

iii. Impact of heuristic accuracy

An accurate heuristic helps prioritize promising routes. An overestimated heuristic can remove A*'s optimality guarantee. An admissible heuristic that never overestimates the remaining optimal cost can preserve A* optimality under standard assumptions. A poor heuristic can lead to inefficient exploration. Straight-line distance is useful for distance, but may not accurately estimate travel time or risk in severe weather.

iv. Impact of dynamic changes and blocked paths

When a path becomes blocked, the drone receives an update from sensors or communication, marks the path unavailable, evaluates alternatives, and replans. GBFS may quickly select an alternative based on distance, while A* also considers accumulated cost. Frequent changes increase computation and can cause delays, so reliable updates, rapid replanning and route safety checks are necessary.

v. Recommended algorithm

Recommendation: A* with dynamic replanning. A* balances actual route cost with estimated remaining cost, making it preferable when travel time, cost and risk matter. A* alone does not guarantee safety; the drone also needs obstacle detection, weather constraints and route validation. An incremental replanning method can reduce repeated computation.

QUESTION 2: SMART CITY TRAFFIC SIGNAL OPTIMIZATION

Problem formulation

Let x represent the signal timings at all intersections. Let W(x) be total waiting time, F(x) be fuel consumption, and C(x) be congestion. The system seeks to minimize a weighted objective while respecting signal safety and timing constraints.

Objective: min $Z(x) = \alpha W(x) + \beta F(x) + \gamma C(x)$

The weights α, β and γ represent the relative importance of each objective. Constraints include minimum and maximum green durations, safe signal transitions, pedestrian crossing times and coordination between intersections.

Why traditional search is inefficient

Uninformed search may explore many possible configurations without considering their quality. With multiple timing options at hundreds of intersections, the number of combinations grows rapidly. Traffic conditions also change, making repeated full searches expensive.

i. How Hill Climbing works and its limitations

Hill Climbing starts with a signal configuration, evaluates its objective value, generates neighboring configurations by changing timings, and moves to a better neighbor. It repeats until no better neighbor is found. For example, extending green time at a congested junction may reduce waiting time.

Limitations: It can get trapped in a local optimum, stop on a plateau, struggle with ridges requiring coordinated changes, fail to guarantee a global optimum, and react poorly if traffic changes faster than optimization can keep up.

ii. Local maxima, plateaus and ridges

Local optimum: A configuration is better than nearby alternatives but worse than a solution elsewhere; for a minimization objective this is a local minimum. For example, improving one junction may shift congestion to a neighboring junction. Plateau: Several neighboring configurations have nearly identical performance, so the algorithm cannot identify a clear direction. Ridge: Improvement requires coordinated changes at several intersections; changing only one signal may not improve traffic flow.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) sometimes accepts a worse solution to escape local optima. It starts at a high temperature and gradually cools. The probability of accepting a worse solution is $P = e^{(-\Delta E/T)}$, where ΔE is the increase in objective cost and T is the current temperature. As temperature falls, the algorithm becomes less likely to accept worse solutions.

Genetic Algorithm (GA) maintains a population of candidate configurations. It uses selection to choose parents, crossover to combine them, mutation to introduce variation, and evaluation to create the next population. Exploring multiple candidates helps it search different regions of the solution space.

iv. Best approach for scalability and adaptability

Recommendation: Genetic Algorithm with simulation-based evaluation and periodic replanning. A GA explores multiple candidate configurations and can coordinate changes across intersections. Simulation, hard safety constraints, parallel evaluation and periodic optimization can improve practicality. However, GA can be computationally expensive and does not guarantee a global optimum; population size and generations should be limited for real-time use.

QUESTION 3: AUTONOMOUS MARS EXPLORATION ROVER

i. Why an online search agent is required

An offline search agent plans using a model available before execution. The Mars rover does not know all craters, rocks or obstacles in advance. An online search agent interleaves planning and action: it moves, gathers sensor information, updates its map and selects its next action. If a crater is discovered on the planned path, the rover can choose another route.

ii. Challenges of partial observability and dynamic environments

Partial observability: Sensors provide limited information; obstacles may be hidden or outside sensor range; readings may contain errors; and terrain traversability may be uncertain. Changing conditions: Dust may reduce visibility, newly discovered obstacles may invalidate a route, and energy, communication delays and hardware limits restrict exploration.

iii. How the rover builds and updates its internal model

The rover can maintain a grid map or graph of locations. It begins with the known starting position and target, senses the surroundings, identifies obstacles and traversable terrain, updates locations as safe, blocked or uncertain, plans an action, executes it, checks the outcome and repeats. It can also track confidence levels for unexplored areas.

iv. Comparison of online, uninformed and informed search

Feature	Online search	Uninformed search	Informed search
Knowledge	Learns during execution	Uses problem definition without heuristic	Uses heuristic guidance
Planning	Interleaves planning and action	Can plan before execution	Can plan before execution
Guidance	Uses observations and updated knowledge	No heuristic guidance	Uses a heuristic estimate
Unknown terrain	Suitable with sensing and replanning	May require a predefined model	May require a model unless adapted

Online search describes how an agent operates with incomplete knowledge, whereas uninformed and informed search describe how search is guided. These categories are not mutually exclusive.

v. Most suitable approach

Recommendation: Online heuristic search with incremental replanning and a local safety mechanism. A* can plan routes when a useful map is available. When new obstacles appear, an incremental method such as D* Lite can reuse prior search information to repair the route. A local obstacle-avoidance system should validate movements, while the planner accounts for energy, terrain risk and communication limitations.

QUESTION 4: UNIVERSITY EXAMINATION TIMETABLE USING CSP

Formulation as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of variables, domains and constraints. The university assigns a valid time slot and hall to each examination while satisfying the requirements.

i. Variables, domains and constraints

Variables: X = {E1, E2, …, En}, where each variable represents one course examination. Domains: D(Ei) contains possible time-slot and hall combinations, such as (T1,H1), (T1,H2) and (T2,H1), where Ti is a time slot and Hi is a hall.

Constraints:

- No overlapping examinations for any student.
- Hall capacity must not be exceeded.
- A hall cannot host multiple examinations simultaneously.
- Subjects with precedence requirements must be scheduled in the required order.
- Faculty must be available for the assigned examination.
- Special accommodations must be satisfied through suitable halls and arrangements.

ii. How backtracking search works

Backtracking assigns values to variables one at a time. The algorithm selects an unscheduled examination, chooses a time and hall, checks constraints, and proceeds if valid. If no valid assignment remains, it undoes a previous assignment and tries another. For example, if students take both Mathematics and Physics, those examinations cannot occupy the same time slot. The search continues until a valid timetable is found or the current search path is shown to fail.

iii. How constraint propagation improves efficiency

Forward checking removes values from the domains of unassigned variables that conflict with a new assignment. If any domain becomes empty, the algorithm backtracks immediately. For example, scheduling Mathematics at 9 AM removes that slot from other examinations that share students.

Additional heuristics: Minimum Remaining Values (MRV) chooses the examination with the fewest legal assignments; the degree heuristic prioritizes examinations constraining many others; and Least Constraining Value (LCV) chooses an assignment that leaves the most options for remaining examinations.

iv. Real-world challenges and scalable strategy

Large timetables involve many students, courses, halls, faculty schedules, accessibility needs and preferences. Constraints may conflict. Recommendation: Backtracking with MRV, the degree heuristic, forward checking and constraint propagation. For large instances, constraint-programming or SAT/MaxSAT solvers can be considered. Soft preferences can be optimized after hard constraints are satisfied.

QUESTION 5: STRATEGIC GAME AI — MINIMAX AND ALPHA-BETA PRUNING

i. How Minimax models the problem

Minimax is used in competitive games with two players having opposing objectives. MAX tries to maximize the evaluation score and MIN tries to minimize it. The game tree represents possible moves and resulting states. The algorithm evaluates terminal states or depth-limited states and propagates values upward.

MAX node: V(n) = max V(c), for children c. MIN node: V(n) = min V(c), for children c.

For example, the AI as MAX chooses the move with the highest guaranteed evaluation, assuming the opponent responds optimally to minimize its score.

ii. Computational challenges of large game trees

If the average branching factor is b and search depth is d, basic Minimax has time complexity O(b^d). With 10 possible moves per state and depth 5, it may examine approximately 100,000 nodes. Challenges include high computation, strict deadlines, memory use, the horizon effect and imperfect heuristic evaluation.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning eliminates branches that cannot affect the final Minimax decision. Alpha (α) is the best value MAX can guarantee so far; beta (β) is the best value MIN can guarantee. A branch can be pruned when α ≥ β.

Example: A MAX node has two MIN children. The first MIN child has leaves 3 and 5, so its value is 3 and α becomes 3. The second MIN child has a first leaf with value 2, so β becomes 2. Because β ≤ α, the remaining children of the second MIN node can be pruned: that branch cannot improve MAX's choice over the first branch.

Advantages: It evaluates fewer nodes, preserves the Minimax decision at the same depth, and permits deeper search within the same time limit. With ideal move ordering, best-case complexity is approximately O(b^(d/2)), compared with O(b^d) for basic Minimax.

iv. Design an evaluation function

A heuristic evaluation estimates how favorable a state is when search cannot reach a terminal state. One possible function is:

E(s) = w1M(s) + w2R(s) + w3T(s) + w4P(s) − w5D(s) − w6C(s)

M is military strength; R is available resources; T is territory controlled; P is strategic position and objective progress; D is damage or losses; and C is resource and maintenance costs. The weights w1–w6 reflect each factor's importance. The function should match the actual rules and goals of the game.

v. Strategy for optimality and real-time performance

Recommendation: Iterative deepening Minimax with Alpha-Beta pruning, move ordering and a time limit. The AI first searches shallowly, orders promising moves, prunes irrelevant branches and increases depth while time remains. Transposition tables can reuse prior evaluations, quiescence search can extend unstable tactical positions, and time management should ensure a legal move is returned before the deadline. The method improves decision quality but does not guarantee a globally optimal move when search is depth-limited and evaluation is approximate.

CONCLUSION

The five scenarios illustrate how AI search and optimization methods address different problems: A* with dynamic replanning for emergency routing; Genetic Algorithms for large-scale traffic optimization; online heuristic search for rover exploration; CSP with backtracking and constraint propagation for examination scheduling; and iterative deepening Minimax with Alpha-Beta pruning for real-time game AI.